

EduGenie: AI-Powered Learning Assistant

Project Description:

EduGenie is a production-ready, full-stack AI educational assistant that helps students, teachers, and self-learners: ask educational questions (with RAG document chat), understand concepts at beginner/intermediate/advanced levels, generate interactive quizzes with scoring and PDF export, summarize notes from text, PDF, or DOCX, receive personalized learning roadmaps, and track progress with analytics, streaks, and badges.

Utilizing Google Gemini 1.5 Pro and LaMini-Flan-T5 models, EduGenie processes user inputs to deliver high-quality educational learning experiences. Built with FastAPI and deployed locally, the platform ensures a secure and responsive UI. With secure API key management via .env and responsible input handling, EduGenie empowers marketers, creators, and businesses to craft compelling social media content with confidence.

Scenarios

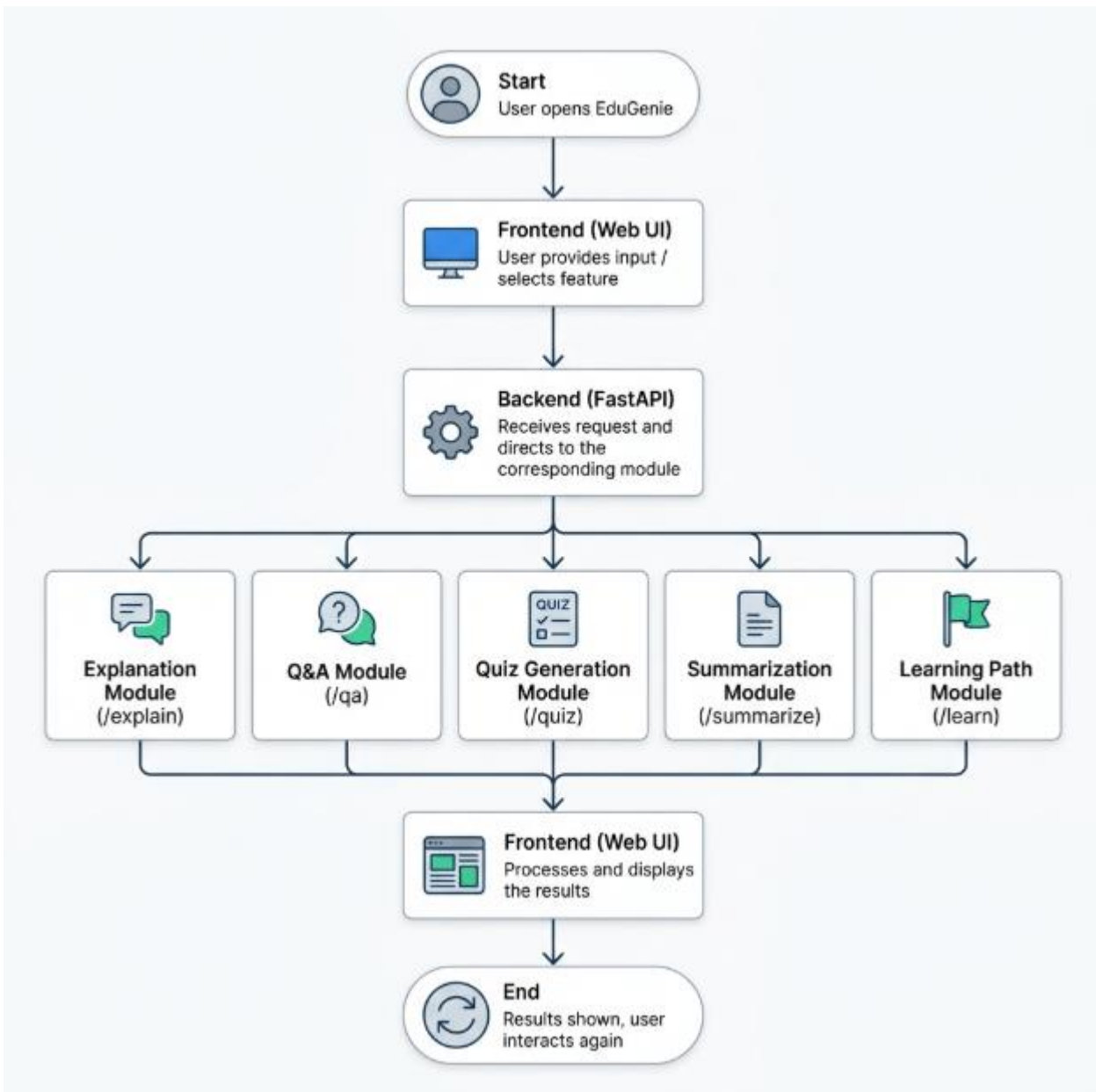
Scenario 1: A high-school student wants to learn Python programming from scratch. They use the Concept Explainer at a Beginner level, receiving simple definitions and everyday analogies to grasp variables and loops.

Scenario 2: A college student uploads a PDF textbook chapter on photosynthesis and uses the RAG Chat to ask specific questions directly from the document, getting citations and context-specific answers.

industry-relevant hashtags, and CTAs designed to drive professional engagement such as demo sign-ups or article reads.

Scenario 3: A self-learner preparing for a job interview generates a 10-question MCQ quiz on SQL databases, takes the test with a timer, and exports their score and results as a PDF for review. deal-seekers, and CTAs like "Shop before it's gone!" to maximize conversion.

Technical Architecture:



Description:

EduGenie utilizes a modular three-tier architecture to deliver rapid, AI-driven educational content. The frontend provides a responsive user interface built with Jinja2, HTML and CSS while the FastAPI backend orchestrates data validation and prompt construction. It interfaces with the Gemini API to generate tailored quizzes, summaries, roadmaps, and Q&A, with secure database storage using local JSON collections.

Pre-requisites:

- FastAPI Framework Knowledge: FastAPI Documentation
- Gemini API Familiarity: Google Gemini API Documentation
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- Development Environment Setup: FastAPI Installation Guide
- Uvicorn for Public Deployment: Uvicorn Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting the appropriate generative AI models and designing the application architecture to support the educational companion's capabilities.

- **Activity 1.1: Generate a Gemini API Key:** Generate a Google Gemini API Key securely from Google AI Studio and configure it in the project's local environment.
- **Activity 1.2: Research and Select Generative AI Models:** Research and configure optimal model configurations: cloud-based gemini-2.5-flash for high-speed streaming Q&A, note summaries, and timeline paths, and a local MBZUAI/LaMini-Flan-T5-783M text pipeline as an optional offline fallback for concept explanation.
- **Activity 1.3: Define the Architecture of the Application:** Draft the layout of the application showing interactions between the Jinja2/HTML frontend UI, FastAPI backend services, local JSON flat-file database collections, and the AI embeddings pipeline.
- **Activity 1.4: Set Up the Development Environment:** Install Python 3.10+, configure the virtual environment, and install dependencies (fastapi, google-generativeai, reportlab, pypdf, python-docx, bcrypt, transformers, etc.) listed in requirements.txt.

Milestone 2: Core Functionalities Development

This milestone focuses on coding the underlying AI business logic modules and organizing request structures.

- **Activity 2.1: Develop Core AI Educational Modules:**
 - **RAG Q&A Module (qa_module.py):** Integrates text embeddings using models/embedding-001 and cosine similarity matches to provide context-grounded document responses.

- **Concept Explainer (explain_module.py):** Formulates prompts to explain topics in Beginner, Intermediate, or Advanced levels, with fallback logic between local LaMini and Gemini API.
- **Quiz Generator (quiz_module.py):** Enforces strict structured JSON array prompts for multi-choice items.
- **Note Summarizer (summary_module.py):** Renders bullet-point or detailed notes from text or parsed documents.
- **Learning Path Generator (learning_path_module.py):** Maps chronological study timeline schedules.
- **Activity 2.2: Implement Request Validation and File Parsing:**
 - Implement Pydantic schema models (schemas.py) for incoming requests.
 - Set up PyPDF and python-docx text extraction methods in pdf_utils.py to parse text contents from uploaded student files, enforcing a 10MB size limit.

Milestone 3: FastAPI Routes and Service Layer Integration

This milestone details setting up the FastAPI server, endpoints, and business service classes.

- **Activity 3.1: Writing the Main Application Entry Point in main.py:** Configure the FastAPI app instance, register CORSMiddleware configuration, mount the /static folder, and include router files.
- **Activity 3.2: Establish Route Handlers:**
 - **routes/pages.py:** Handles page routing to render Jinja2 pages (Landing, Login, Signup, Dashboard, Q&A, Explainer, Quiz, Summary, Learning Path, Bookmarks, and Settings).
 - **routes/api.py:** Manages API requests, user credentials validation, password hashing (bcrypt), JWT token cookie creation, and PDF document downloads.
- **Activity 3.3: Implement the Service Layer:** Write modular services (AuthService, HistoryService, BookmarkService, AnalyticsService, SettingsService, and RAGService) to read and write records in the local JSON database files.
- **Activity 3.4: Configure PDF Export Utilities:** Write export_utils.py using reportlab.pdfgen.canvas to draw cleanly formatted, downloadable PDF summaries, learning roadmaps, and quiz result sheets on-the-fly.

Milestone 4: Frontend Development

In this milestone, the visual glassmorphic dashboard interface is constructed and connected to the backend.

- **Activity 4.1: Design and Develop the Glassmorphic UI Layout:** Develop a responsive dark-themed dashboard interface using CSS variables (variables.css), base structural styles (base.css), and dashboard layout sheets (dashboard.css). Configure dynamic light/dark mode theme toggling.
- **Activity 4.2: Create Server-Rendered Jinja2 Templates:** Create modular HTML files under the templates/ folder (such as the base navigation structure base_dashboard.html and specific page views like quiz.html and ask.html) to display personalized user data.
- **Activity 4.3: Implement Asynchronous JavaScript Controllers:** Write client-side JS files (ask.js, quiz.js, summary.js, learning_path.js) to make async Fetch requests to endpoints, updating the UI dynamically (e.g., rendering timers, rendering timeline paths, rendering quiz scoreboards, and handling text-to-speech toggles).

Milestone 5: Deployment

This milestone details compiling the application, configuring server environments, and testing the request flow.

- **Activity 5.1: Prepare the Server Environment:** Build directory structures for database storage and uploads, copy .env.example to .env, and configure host bindings (0.0.0.0) and default ports (8000).
- **Activity 5.2: Local Verification:** Launch the local ASGI server using Uvicorn via python run.py. Navigate to <http://localhost:8000>, register a demo account, and verify that API calls stream responses correctly.

Milestone 6: Conclusion and Future Scope

- **Activity 6.1: Conclusion.**
- **Activity 6.2: Future Scope**

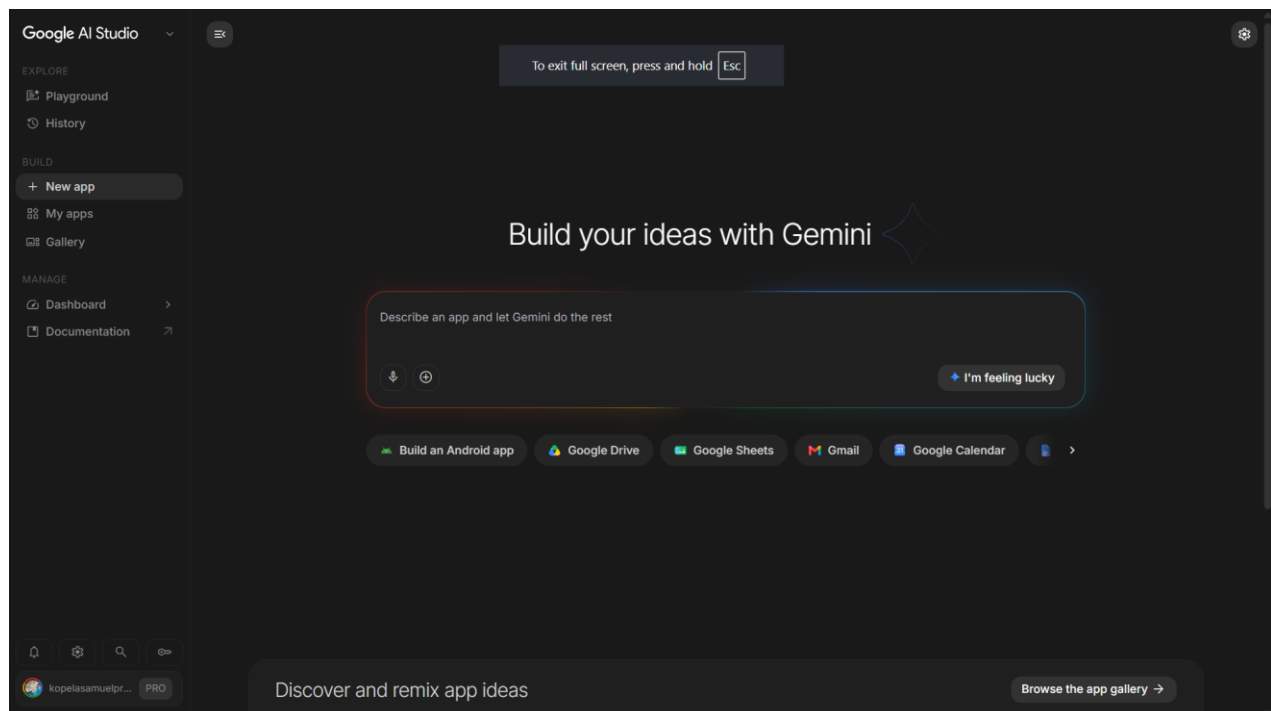
Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting the appropriate generative AI models for the EduGenie educational learning assistant's needs. This involves researching the capabilities and performance of cloud-based models (Google Gemini API via the official google-generativeai SDK) and local offline models (LaMini-Flan-T5), ensuring that the selected models align with the application's objectives of generating Q&A responses, scaled concept explanations, interactive quizzes, note summaries, and personalized learning roadmaps.

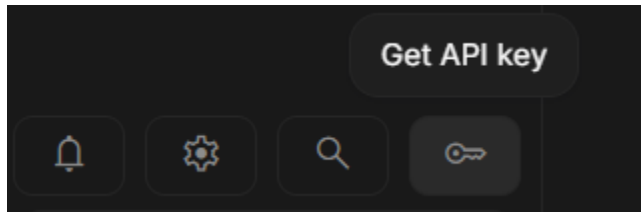
Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini Large Language Models (LLM), you need a valid Gemini API key. Follow the steps below to create one and configure it securely in the project environment:

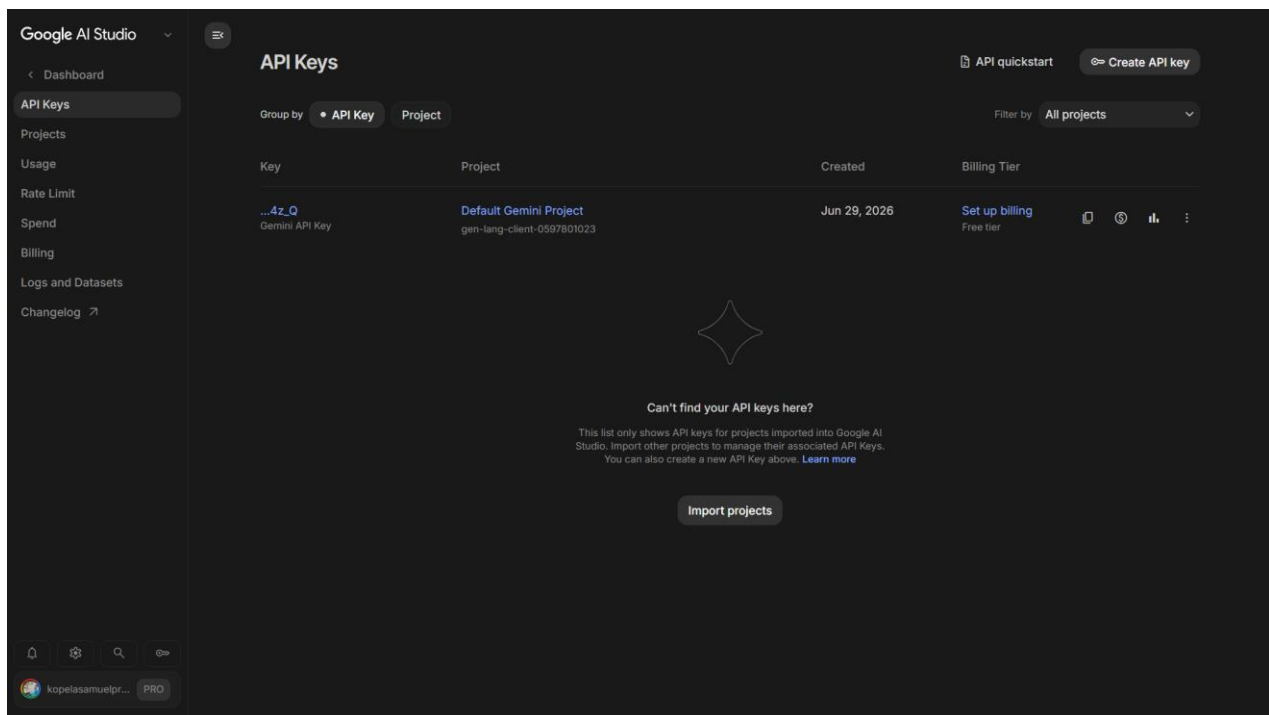
- Step 1 — Create a Google AI Studio Account: Visit <https://aistudio.google.com/> and sign in using your Google account.



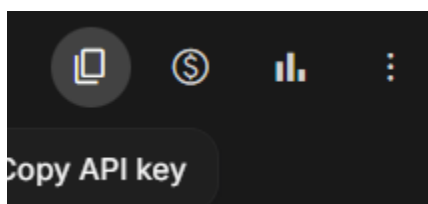
- Step 2 — Navigate to Get API Key: Once logged in, click on the "Get API Key" button in the left-hand sidebar menu.



- Step 3 — Create a New API Key: Click the "Create API Key" button. You can choose to generate a key in a new Google Cloud Project or select an existing Google Cloud Project.



- Step 4 — Copy Your API Key: Once the key is generated, copy it immediately and store it in a safe place. You can view or copy it from the key list at any time.



- Step 5 — Add the Key to Your Project: Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GEMINI_API_KEY=your_copied_gemini_api_key_here  
SECRET_KEY=your_custom_session_secret_key_here
```

- Step 6 — Verify the Key is Loaded: The application loads this key automatically at startup using python-dotenv. In app/config.py, configuration is loaded:

```
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
```

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")

And in app/ai/gemini_client.py, the official Gemini SDK is configured:

```
import google.generativeai as genai  
from app.config import GEMINI_API_KEY, GEMINI_MODEL  
  
class GeminiClient:  
    def _ensure_configured(self):  
        if not self._configured:  
            genai.configure(api_key=self.api_key)  
            self._model = genai.GenerativeModel(GEMINI_MODEL)  
            self._configured = True
```

Important — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
# Environments  
.env  
.env.*  
venv/  
storage/
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the criteria researched to select the optimal models for the project:

- **Understand the Project Requirements:** Review the specific needs of the EduGenie application, focusing on RAG (Retrieval-Augmented Generation) document chat, multi-level concept scaling, quiz construction, note summaries, timeline-based study roadmaps, and PDF exports.
 - **Explore Model Documentation:** Examine the available Google Gemini models (Gemini 1.5 Pro and Gemini 2.5 Flash), focusing on their context windows, inference speed, structure capabilities (like returning JSON objects), and pricing.
 - **Explore Local Models:** Research lightweight open-source models (such as MBZUAI/LaMini-Flan-T5-783M) to act as a local, offline fallback for concept scaling when internet access or API usage is limited.
 - **Select the Optimal Model:** Choose gemini-2.5-flash as the default cloud model for its balance of high creative quality, fast text generation speed, structural JSON generation, and native document embedding capabilities. Set MBZUAI/LaMini-Flan-T5-783M locally via Hugging Face transformers as an optional local fallback.
-

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, detailing the interactions between the frontend, FastAPI backend, local JSON database layer, and AI integration services.
- **Detail Frontend Functionality:** Outline how users register, login, view study analytics metrics on the home dashboard, select difficulty levels (Beginner, Intermediate, Advanced) in the Concept Explainer, submit questions to the Q&A engine, upload PDF/Docx files for RAG chat and summaries, and download exported PDF documents.
- **Outline Backend Responsibilities:** Specify how the FastAPI backend manages route handlers (/api endpoints and server-rendered HTML pages via Jinja2), handles user sessions with secure JWT-signed browser cookies, executes password security checkups (bcrypt), chunks and embeds uploaded documents, runs cosine-similarity calculations to pull RAG context, and compiles programmatic PDFs using reportlab.
- **Describe AI Integration Points:** Define how AI modules (qa_module, explain_module, quiz_module, summary_module, learning_path_module) format templates/prompts, execute API streaming calls using gemini_client, handle fallback checks to the local LaMiniClient pipeline, and return structured educational outputs to the frontend.

Activity 1.4: Set Up the Development Environment

- Install Python and Pip: Ensure Python 3.10+ is installed on your machine along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies:

`python -m venv venv`

```
PS D:\sury_made\p8.1> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\sury_made\p8.1\venv\Scripts\Activate.ps1)
(venv) PS D:\sury_made\p8.1>
```

Activate the Virtual Environment:

- Windows (CMD): `venv\Scripts\activate.bat`
- Windows (PowerShell): `venv\Scripts\Activate.ps1`
- macOS / Linux: `source venv/bin/activate`
- Install FastAPI & Dependencies: Use pip to install FastAPI, the Google Generative AI SDK, and all required packages from requirements.txt:

`pip install -r requirements.txt`

(Dependencies include: fastapi, uvicorn[standard], jinja2, google-generativeai, python-dotenv, pypdf, python-docx, reportlab, bcrypt, python-jose, transformers, torch)

```
(venv) PS D:\sury_made\p8.1> pip install -r requirements.txt
Requirement already satisfied: httpcore<1.* in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from httpx==0.28.0->-r requirements.txt (line 18)) (1.0.9)
Requirement already satisfied: lxml>=3.1.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from python-docx==1.1.2->-r requirements.txt (line 14)) (6.0.0)
Requirement already satisfied: pillow>=9.0.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from reportlab==4.2.5->-r requirements.txt (line 15)) (12.0.0)
Requirement already satisfied: dnspython>=2.0.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from email-validator==2.2.0->-r requirements.txt (line 17)) (2.8.0)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from pydantic==2.11.0->-r requirements.txt (line 18)) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from pydantic==2.11.0->-r requirements.txt (line 18)) (2.46.4)
Requirement already satisfied: colorama in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from click==7.0->-r requirements.txt (line 2)) (0.4.6)
Requirement already satisfied: cffi>=2.0.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from cryptography==38.0.3->-r requirements.txt (line 7)) (2.0.0)
Requirement already satisfied: pypdf in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from google-generativeai==0.6.4->-r requirements.txt (line 7)) (3.0.0)
Requirement already satisfied: six>=1.9.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from ecdsa==0.15->-r requirements.txt (line 5)) (1.17.0)
Requirement already satisfied: httptools>=0.8.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from uvicorn[standard]>=0.34.0->-r requirements.txt (line 2)) (0.8.0)
Requirement already satisfied: pyyaml>=5.1 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from uvicorn[standard]>=0.34.0->-r requirements.txt (line 2)) (6.0.3)
Requirement already satisfied: watchfiles>=0.20 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from uvicorn[standard]>=0.34.0->-r requirements.txt (line 2)) (1.2.0)
Requirement already satisfied: websockets>=10.4 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from uvicorn[standard]>=0.34.0->-r requirements.txt (line 2)) (16.0)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in c:\users\sury_made\appdata\local\programs\python\python314\lib\site-packages (from google-api-python-client->-r requirements.txt (line 1)) (0.19.0)
```

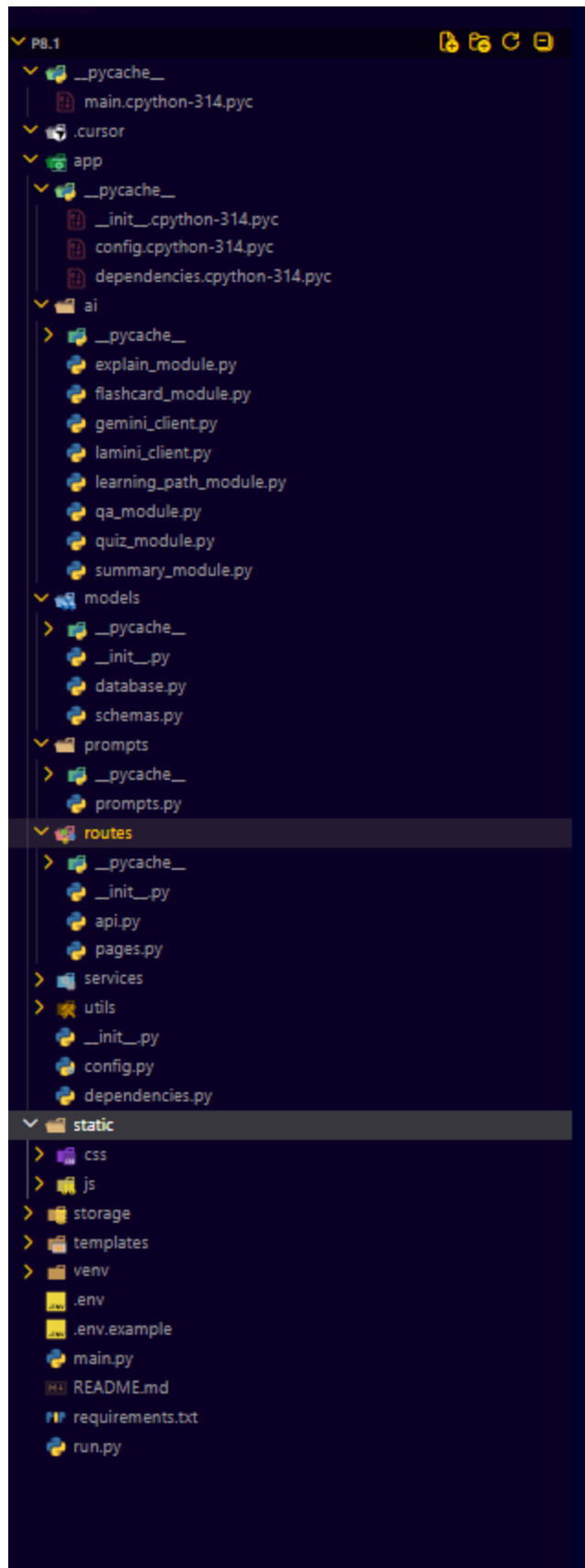
"IF THE REQUIREMENTS ARE ALREADY INSTALLED THE TERMINAL LOOKS LIKE THIS"

Configure the API Key: Create a `.env` file in the project root and add your Google Gemini API key:

GEMINI_API_KEY=your_gemini_api_key_here

SECRET_KEY=your_session_secret_key_here

- Set Up Application Structure: Verify the directory structure contains the following key folders:



Milestone 2: Core Functionalities Development

In this milestone, we focus on developing the core AI educational features and implementing the FastAPI backend. This involves creating separate modular Python components for each study tool and structuring the server routes, schemas, and file parsing operations to handle client inputs, verify security, and integrate LLM API interfaces.

Activity 2.1: Develop Core AI Educational Modules

Develop individual, modular AI modules under the `app/ai/` directory. Each module constructs specific system prompt instructions, handles API requests using the Gemini Client, and structures the AI responses:

- Question Answering & RAG Document Chat Module (`app/ai/qa_module.py`):
 - Core functionality: Accepts direct user questions or uploads document reference parameters.
 - Streams conversational responses chunk-by-chunk using Gemini 1.5 Pro to provide a responsive interface.
 - Integrates with the RAG service (`app/services/rag_service.py`), which uses the `models/embedding-001` model to create embeddings for document text chunks, runs cosine similarity calculations against query vectors, and injects the matching text chunks as contextual information inside the Gemini prompt.
- Concept Explainer Module (`app/ai/explain_module.py`):
 - Core functionality: Generates detailed explanations for topics scaled to three difficulty levels: Beginner (focusing on simple definitions and everyday analogies), Intermediate (explaining core concepts and workflows), or Advanced (delving into details and technical definitions).
 - Integrates with the local model client (`app/ai/lamini_client.py`). If the local Hugging Face transformers pipeline is loaded with MBZUAI/LaMini-Flan-T5-783M, it processes the explanation locally. Otherwise, it falls back to the Google Gemini API to prevent app failure.

- Quiz Generator Module (app/ai/quiz_module.py):
 - Core functionality: Generates multiple-choice quiz questions based on the topic, difficulty level, and requested quantity.
 - Instructs the Gemini API via a strict system prompt to return a valid JSON list containing question texts, options arrays, correct indices, and explanation strings.
 - Note Summarizer Module (app/ai/summary_module.py):
 - Core functionality: Processes raw text strings or parsed files. Renders summaries as Bulleted Points or Detailed Study Notes depending on user specifications.
 - Personalized Learning Path Module (app/ai/learning_path_module.py):
 - Core functionality: Generates study roadmaps based on a topic, student's level, weekly hours, and learning goal. The output is structured as a detailed weekly study plan.
 - Flashcard Generator Module (app/ai/flashcard_module.py):
 - Core functionality: Instantly generates a deck of study flashcards (front and back text items) on a specified topic.
-

Activity 2.2: Implement the FastAPI Backend

Structure the backend application layers to handle routing, request validation, authentication, and file parsing:

1. Define Routes in FastAPI

Configure routing handlers in the app/routes/ directory to separate HTML page renders from API requests:

- Page Rendering Routes (app/routes/pages.py):

- Set up endpoints to serve server-rendered Jinja2 templates for the public landing page (/), authentication pages (/login, /signup), the central hub (/dashboard), and individual tool dashboards (/ask, /explain, /quiz, /summary, /learning-path, /history, /settings, /profile).
- REST API Routes (app/routes/api.py):
 - Set up POST/GET API endpoints (/api/signup, /api/login, /api/ask, /api/ask/stream, /api/explain, /api/quiz, /api/summary, /api/learning-path, /api/upload, /api/bookmarks, /api/settings, /api/profile, and /api/export/*-pdf).

```
router = APIRouter()
templates = Jinja2Templates(directory=str(TEMPLATES_DIR))

def _ctx(user: dict = None, **extra):
    base = {
        "app_name": APP_NAME,
        "user": user,
    }
    base.update(extra)
    return base

@router.get("/", response_class=HTMLResponse)
async def landing(request: Request, access_token: str = Cookie(None)):
    user_id = await get_current_user_id(request, access_token)
    user = auth_service.get_user(user_id) if user_id else None
    return templates.TemplateResponse(request, "landing.html", _ctx(user))

@router.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    return templates.TemplateResponse(request, "login.html", _ctx())

@router.get("/signup", response_class=HTMLResponse)
async def signup_page(request: Request):
    return templates.TemplateResponse(request, "signup.html", _ctx())

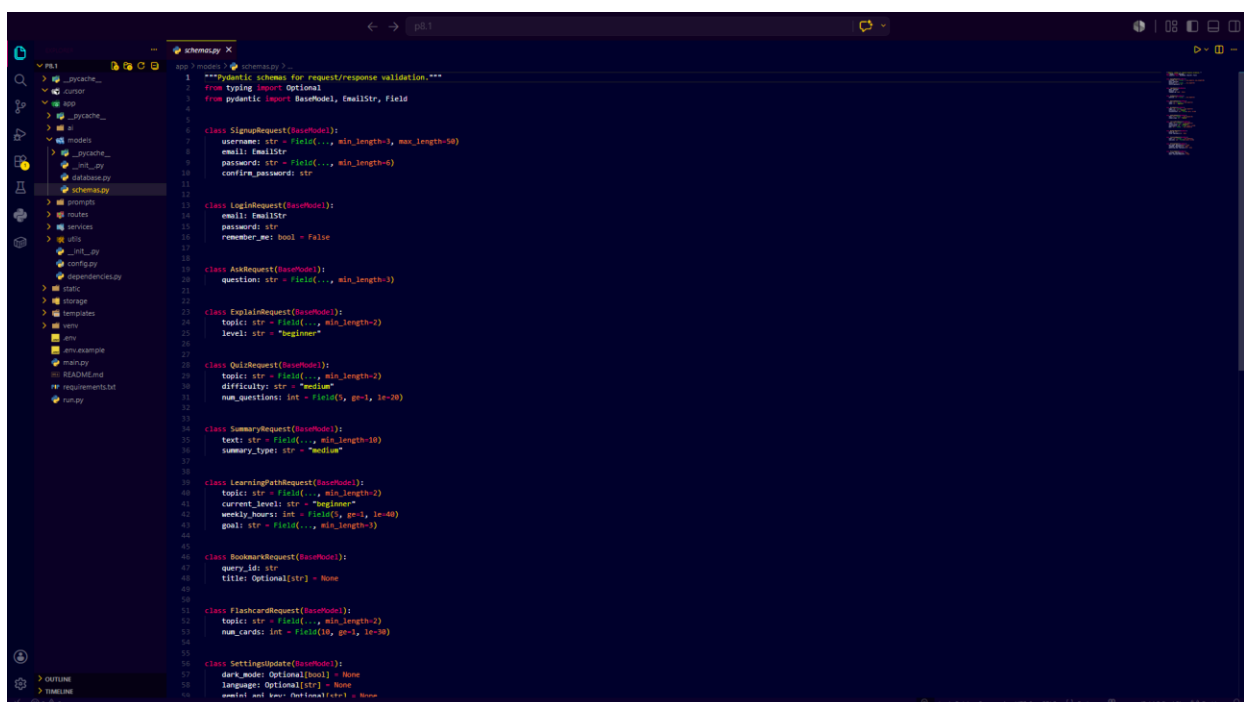
@router.get("/logout")
async def logout():
    response = RedirectResponse(url="/", status_code=302)
    response.delete_cookie("access_token")
    return response

@router.get("/dashboard", response_class=HTMLResponse)
async def dashboard(request: Request, user: dict = Depends(require_user)):
    data = analytics_service.get_dashboard_data(user["UserID"])
    settings = settings_service.get(user["UserID"])
    return templates.TemplateResponse(
        request,
        "dashboard/home.html",
        _ctx(user, stats=data, settings=settings, active="home"),
    )
```

2. Process and Validate User Input

- Pydantic Validation Schemas (app/models/schemas.py):

- Create strict request validation models (e.g., AskRequest, ExplainRequest, QuizRequest, SummaryRequest, LearningPathRequest, SignupRequest, LoginRequest).
- Enforce bounds on user parameters (e.g., email validation, matching passwords, valid difficulty levels, maximum text limits).
- JWT Cookie Authentication & Password Security:
 - Configure session dependencies (app/dependencies.py) to read browser HTTP-only cookies (access_token).
 - Decode JWT credentials to authorize user access, verify password matching using bcrypt during sign-in, and validate active sessions against local JSON database collections.
- File Upload Handling:
 - Configure multipart form uploads in the /api/upload endpoint using FastAPI's UploadFile module.
 - Extract text contents from files (pypdf parser for PDF uploads, python-docx parser for DOCX uploads) while enforcing a strict 10MB maximum size limit.

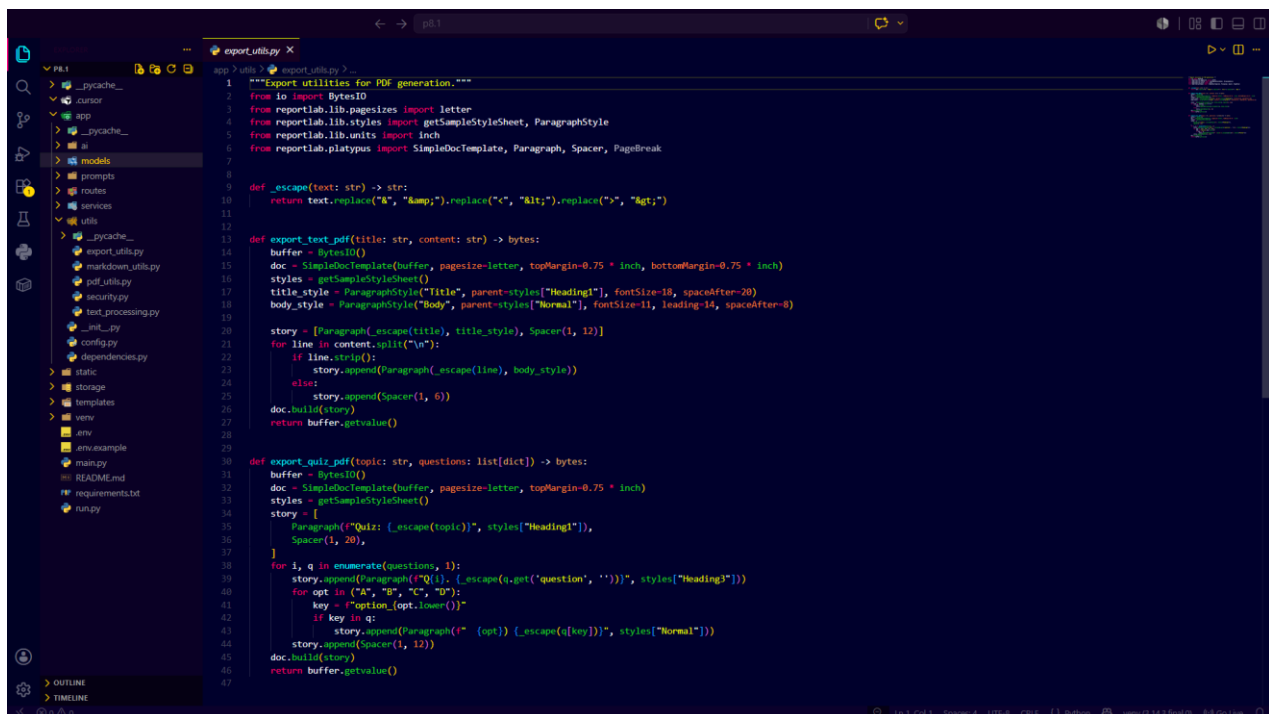


```

1  """Pydantic schemas for request/response validation."""
2  from typing import Optional
3  from pydantic import BaseModel, EmailStr, Field
4
5  class SignupRequest(BaseModel):
6      username: str = Field(..., min_length=3, max_length=50)
7      email: EmailStr
8      password: str = Field(..., min_length=6)
9      confirm_password: str
10
11  class LoginRequest(BaseModel):
12      email: EmailStr
13      password: str
14      remember_me: bool = False
15
16  class AskRequest(BaseModel):
17      question: str = Field(..., min_length=3)
18
19  class ExplainRequest(BaseModel):
20      topic: str = Field(..., min_length=2)
21      level: str = "beginner"
22
23  class QuizRequest(BaseModel):
24      topic: str = Field(..., min_length=2)
25      difficulty: str = "medium"
26      num_questions: int = Field(..., ge=1, le=20)
27
28  class SummaryRequest(BaseModel):
29      text: str = Field(..., min_length=10)
30      summary_type: str = "medium"
31
32  class LearningPathRequest(BaseModel):
33      topic: str = Field(..., min_length=2)
34      current_level: str = "beginner"
35      weekly_hours: int = Field(..., ge=1, le=40)
36      goal: str = Field(..., min_length=3)
37
38  class BookmarkRequest(BaseModel):
39      query_id: str
40      title: Optional[str] = None
41
42  class FlashcardRequest(BaseModel):
43      topic: str = Field(..., min_length=2)
44      num_cards: int = Field(..., ge=1, le=30)
45
46  class SettingsUpdate(BaseModel):
47      dark_mode: Optional[bool] = None
48      language: Optional[str] = None
49      email: Optional[EmailStr] = None
  
```


3. Integrate AI API and Local Pipelines

- Google Generative AI SDK Interactions:
 - Use the Gemini Client to send system-defined prompts, specify model parameters (like temperature), and manage asynchronous streaming responses (StreamingResponse) using Python generators.
- Local Transformers Pipeline Management:
 - Integrate Hugging Face transformers execution logic. Load text generation pipelines locally and handle fallback checks to the Gemini API if memory allocation errors occur.
- Database Logs & File Exports:
 - Log queries, replies, and time timestamps in the local JSON database via HistoryService.
 - Compile programmatic, print-ready PDF binary downloads from study guides, summaries, and quizzes using reportlab.pdfgen.canvas.



```

1  """Export utilities for PDF generation."""
2  from io import BytesIO
3  from reportlab.lib.pagesizes import letter
4  from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
5  from reportlab.lib.units import inch
6  from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak
7
8
9  def _escape(text: str) -> str:
10     return text.replace("&", "&").replace("<", "<").replace(">", ">")
11
12
13  def export_text_pdf(title: str, content: str) -> bytes:
14     buffer = BytesIO()
15     doc = SimpleDocTemplate(buffer, pagesize=letter, topMargin=0.75 * inch, bottomMargin=0.75 * inch)
16     styles = getSampleStyleSheet()
17     title_style = ParagraphStyle("Title", parent=styles["Heading1"], fontSize=18, spaceAfter=20)
18     body_style = ParagraphStyle("Body", parent=styles["Normal"], fontSize=11, leading=14, spaceAfter=8)
19
20     story = [Paragraph(_escape(title), title_style), Spacer(1, 12)]
21     for line in content.split("\n"):
22         if line.strip():
23             story.append(Paragraph(_escape(line), body_style))
24         else:
25             story.append(Spacer(1, 6))
26     doc.build(story)
27     return buffer.getvalue()
28
29
30  def export_quiz_pdf(topic: str, questions: list[dict]) -> bytes:
31     buffer = BytesIO()
32     doc = SimpleDocTemplate(buffer, pagesize=letter, topMargin=0.75 * inch)
33     styles = getSampleStyleSheet()
34     story = [
35         Paragraph(f"Quiz: {_escape(topic)}", styles["Heading1"]),
36         Spacer(1, 20),
37     ]
38     for i, q in enumerate(questions, 1):
39         story.append(Paragraph(f"Q{i}: {_escape(q.get('question', ''))}", styles["Heading1"]))
40         for opt in ("A", "B", "C", "D"):
41             key = f"option_{opt.lower()}"
42             if key in q:
43                 story.append(Paragraph(f"({opt}) {_escape(q[key])}", styles["Normal"]))
44         story.append(Spacer(1, 12))
45     doc.build(story)
46     return buffer.getvalue()
47
  
```

Milestone 3: Main Application configurations & Service Layer Integration

Milestone 3 focuses on creating and refining the core configuration logic in main.py and establishing the underlying service layers. In this milestone, you will set up the main FastAPI router inclusions, configure CORS middleware and static file paths, establish service engines for authentication, history tracking, and RAG document indexing, configure custom AI prompts, and implement robust JSON parsing and PDF export operations.

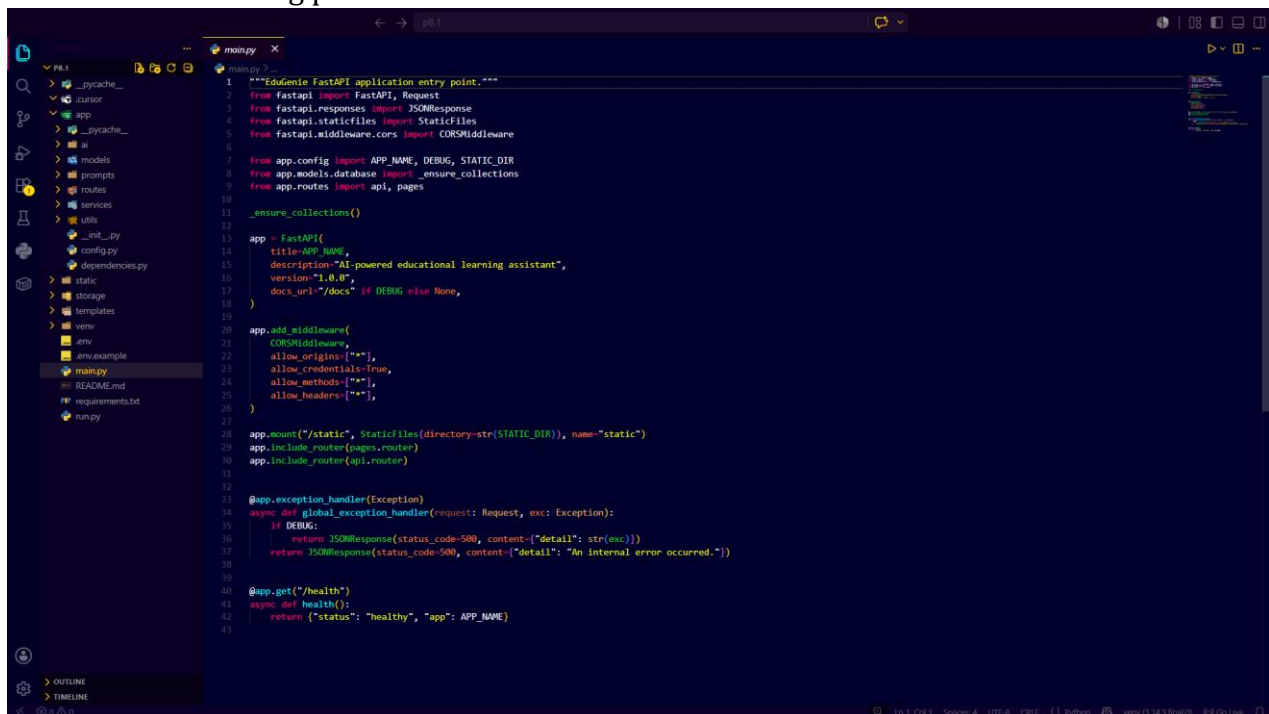
Activity 3.1: Writing the Main Application Logic in main.py

Define the entry point and middle layers of the EduGenie application in the root main.py file:

- **FastAPI Application Initialization:** Create the FastAPI instance, configure document routes (available in development mode), and register global CORSMiddleware.
- **Mounting Assets & Including Routers:** Mount the /static directory to serve modular stylesheets and JS files. Include the pages router (pages.router) and REST API router (api.router) to separate views:

```
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
app.include_router(pages.router)
app.include_router(api.router)
```

Global Error Management: Implement a catch-all exception handler to intercept exceptions globally and return standardized JSON error responses, preventing server crashes during production.

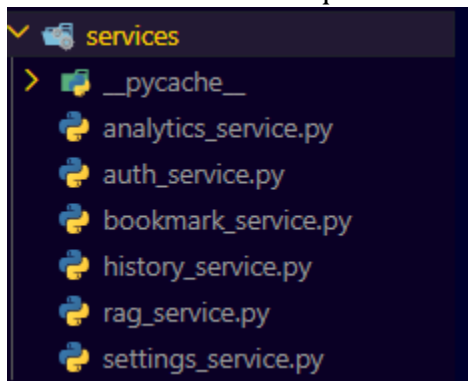


```
1 """EduGenie FastAPI application entry point."""
2 from fastapi import FastAPI, Request
3 from fastapi.responses import JSONResponse
4 from fastapi.staticfiles import StaticFiles
5 from fastapi.middleware.cors import CORSMiddleware
6
7 from app.config import APP_NAME, DEBUG, STATIC_DIR
8 from app.models.database import ensure_collections
9 from app.routes import api, pages
10
11 ensure_collections()
12
13 app = FastAPI(
14     title=APP_NAME,
15     description="AI-powered educational learning assistant",
16     version="1.0.0",
17     docs_url="/docs" if DEBUG else None,
18 )
19
20 app.add_middleware(
21     CORSMiddleware,
22     allow_origins=["*"],
23     allow_credentials=True,
24     allow_methods=["*"],
25     allow_headers=["*"],
26 )
27
28 app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
29 app.include_router(pages.router)
30 app.include_router(api.router)
31
32
33 @app.exception_handler(Exception)
34 async def global_exception_handler(request: Request, exc: Exception):
35     if DEBUG:
36         return JSONResponse(status_code=500, content={"detail": str(exc)})
37     return JSONResponse(status_code=500, content={"detail": "An internal error occurred."})
38
39
40 @app.get("/health")
41 async def health():
42     return {"status": "healthy", "app": APP_NAME}
```

Activity 3.2: Implementing Service Layer Engines

Instead of putting all business logic inside route handlers, establish individual service classes under `app/services/` to manage database modifications and state updates:

- Authentication Service (`app/services/auth_service.py`):
 - Handles user registration, verifies credentials, hashes passwords using `bcrypt`, and generates JWT access tokens.
- History Service (`app/services/history_service.py`):
 - Logs student queries (type, prompt, timestamp) and saves AI responses in the local JSON database file.
- Bookmark Service (`app/services/bookmark_service.py`):
 - Handles marking, lists, and deleting saved AI responses.
- Analytics Service (`app/services/analytics_service.py`):
 - Tracks cumulative usage stats (questions asked, quizzes completed, summaries generated).
 - Calculates daily study streaks and triggers badge achievements when specific milestones are reached.
- RAG Vector Service (`app/services/rag_service.py`):
 - Handles document text chunking and embeds text vectors using `models/embedding-001`.
 - Computes cosine similarity scores to retrieve the `top_k` most relevant chunks for user queries.



Activity 3.3: Prompt Engineering and JSON Extraction

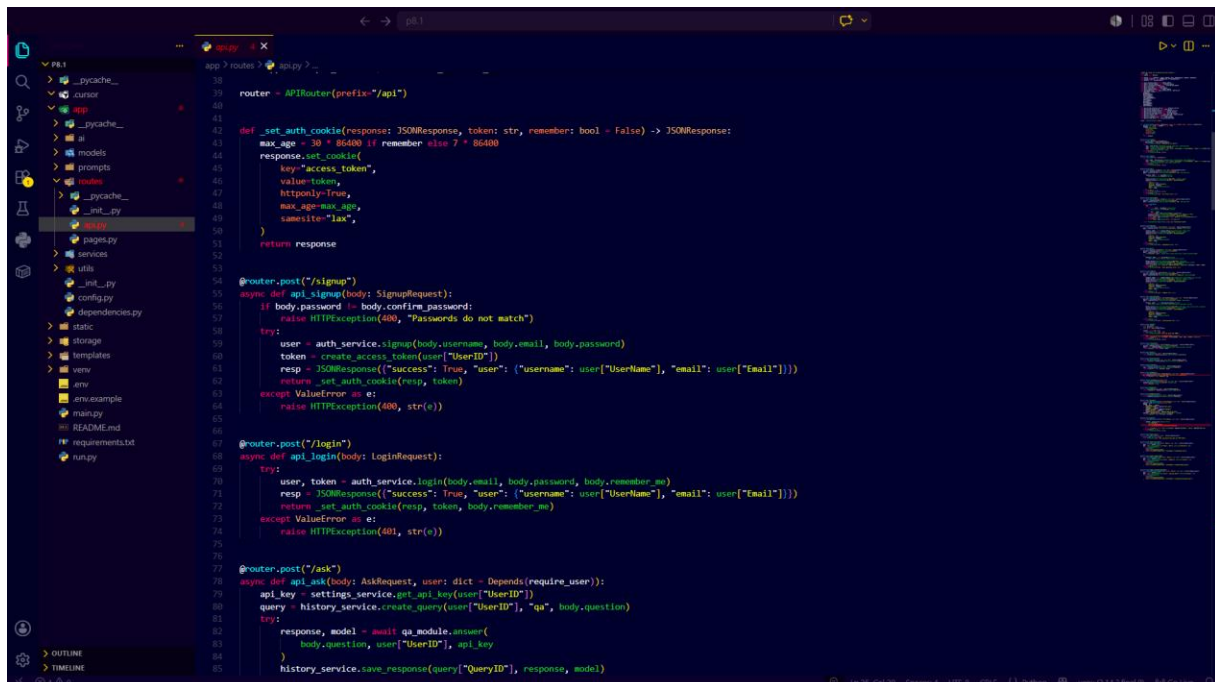
- Educational System Prompts (`app/prompts/prompts.py`):
 - Configure specific system templates like `QUIZ_PROMPT`, `EXPLAIN_PROMPT`, and `QA_SYSTEM` that instruct the model to behave as an expert tutor, use analogies, and outputs responses in Markdown or structured JSON arrays.
- Robust JSON Array Parser (`_parse_json` in `quiz_module.py`):
 - Implement extraction using regular expressions (`r"\[[\s\S]*\]"`) to locate the start and end of JSON arrays in the model's text output.
 - Attempt standard parsing (`json.loads()`).
 - Implement key normalization (`_normalize_q`) to map variations like `option_a` vs. `optionA` or `correct_option` vs. `correctOption` into a single

standardized dictionary key, falling back to clean placeholders if the parse fails.

```
def _parse_json(self, text: str) -> list[dict]:
    match = re.search(r"\\[[\\s\\S]*\\]", text)
    if match:
        try:
            data = json.loads(match.group())
            if isinstance(data, list):
                return [self._normalize_q(q) for q in data]
        except json.JSONDecodeError:
            pass
    return [self._fallback_question(text)]
```

Activity 3.4: Programmatic PDF Export Services

- Dynamic Study PDF compilation (app/utils/export_utils.py):
 - Configure reportlab.pdfgen.canvas utilities to create programmatic, download-ready PDF binary streams for quizzes, concept summaries, and learning roadmaps on-the-fly.
 - Set margins, fonts, colors, and line wraps, and return the files as FastAPI StreamingResponse objects with standard attachment; filename=... headers.



```
router = APIRouter(prefix="/api")

def _set_auth_cookie(response: JSONResponse, token: str, remember: bool = False) -> JSONResponse:
    max_age = 30 * 86400 if remember else 7 * 86400
    response.set_cookie(
        key="access_token",
        value=token,
        httponly=True,
        max_age=max_age,
        samesite="lax",
    )
    return response

@router.post("/signup")
async def api_signup(body: SignupRequest):
    if body.password != body.confirm_password:
        raise HTTPException(400, "Passwords do not match")
    try:
        user = auth_service.signup(body.username, body.email, body.password)
        token = create_access_token(user["UserID"])
        resp = JSONResponse({"success": True, "user": {"username": user["UserName"], "email": user["Email"]}})
        return _set_auth_cookie(resp, token)
    except ValueError as e:
        raise HTTPException(400, str(e))

@router.post("/login")
async def api_login(body: LoginRequest):
    try:
        user, token = auth_service.login(body.email, body.password, body.remember_me)
        resp = JSONResponse({"success": True, "user": {"username": user["UserName"], "email": user["Email"]}})
        return _set_auth_cookie(resp, token, body.remember_me)
    except ValueError as e:
        raise HTTPException(401, str(e))

@router.post("/ask")
async def api_ask(body: AskRequest, user: dict = Depends(require_user)):
    api_key = settings_service.get_api_key(user["UserID"])
    query = history_service.create_query(user["UserID"], "qa", body.question)
    try:
        response, model = mail_qa_module.answer(
            body.question, user["UserID"], api_key
        )
        history_service.save_response(query["QueryID"], response, model)
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, responsive, and visually appealing interface for the EduGenie application. This involves designing a premium dark glassmorphism dashboard layout with CSS variable configurations, building server-rendered templates using Jinja2 inheritance, and writing client-side JavaScript controllers to manage asynchronous REST calls, audio speech features, and dynamic content rendering.

Activity 4.1: Designing and Developing the User Interface

1. Set Up the Jinja2 Template Hierarchy

- Base Scaffold Template (templates/base.html):
 - Establish the HTML head tag details, integrate Font Awesome icon packs, and import clean typography (Inter Google Fonts).
- Dashboard Shell Sidebar Layout (templates/dashboard/base_dashboard.html):
 - Design a sidebar navigation layout to link the different modules (Home, Q&A Chat, Concept Explainer, Quiz Play, Note Summarizer, Learning Path, Query History, Settings, and Profile).
 - Build user profile indicators, sign-out actions, and theme toggle buttons.
- Component View Files:
 - Build individual module views (home.html, ask.html, quiz.html, summary.html, learning_path.html) that inherit the sidebar scaffold, utilizing HTML5 elements to structure the layout.

2. Design a Responsive Glassmorphic Layout Using CSS

Organize styles into modular CSS files under static/css/ to ensure maintainability:

- Global Styles & CSS Variables (variables.css & base.css):

- Set up CSS custom properties (color palettes, shadows, borders, transitions) and global resets.
- Dashboard Layout and Themes (dashboard.css & dark-mode.css):
 - Create a dark-mode theme utilizing glassmorphism cards (translucent backgrounds with backdrop-filter: blur()).
 - Implement flexbox sidebars and responsive grid systems to organize dashboards, sidebars, forms, and results containers.
 - Define media queries to scale the interface across mobile devices, tablets, and desktop displays.
- Dynamic Animations (animations.css):
 - Add custom animations for background blur blobs, fade-in transitions, and slide animations.

3. Construct Custom UI Components

- Q&A Chat Containers: Clean conversational chat dialogs displaying user messages (right-aligned) and assistant replies (left-aligned) as separate glass cards.
 - Interactive Quiz Cards: A questionnaire interface showing multiple-choice selectors (Options A, B, C, D) that respond visually to hover, select, and grading states.
 - Hashtag/Chip and Timelines: Render timeline lists for learning roadmaps, summary chips, and notification toast popups.
-

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Write specialized JavaScript controllers under static/js/ to run asynchronous API interactions and render results:

1. Asynchronous Event Controllers

- Asynchronous Fetch Requests (ask.js, quiz.js, explain.js, summary.js, learning_path.js):

- Attach submit listeners to forms to prevent default browser reloads.
- Gather form inputs, package details into JSON bodies, and send asynchronous POST requests to backend endpoints via the browser Fetch API.
- Interactive Loader States:
 - During request processing, disable the submit inputs, add loading classes to buttons, and show animated CSS loader spinners to provide feedback.
- Speech Integration (speech.js):
 - Connect Web Speech API interfaces to support voice-to-text queries and text-to-speech feedback.

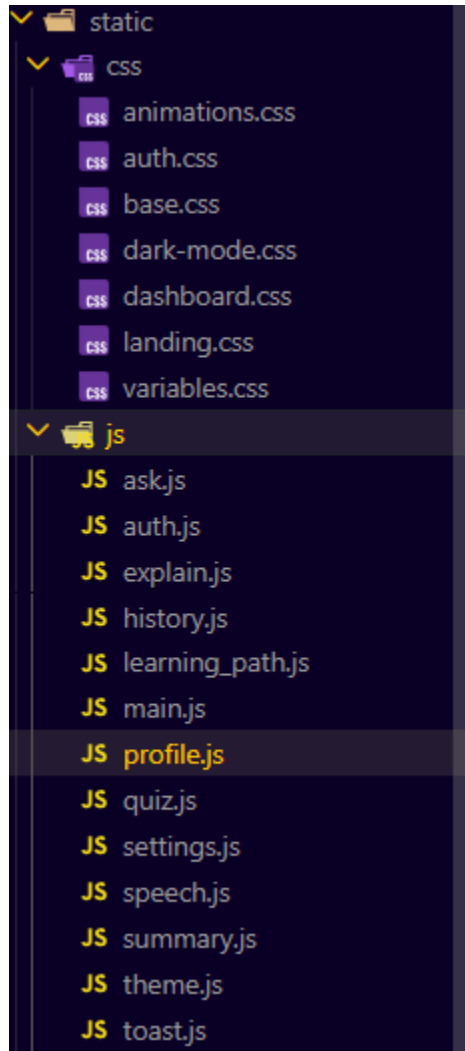
2. Dynamic DOM Rendering and Client Updates

- Dynamic Results Injections:
 - Implement functions to clear previous results containers and inject HTML nodes on response success.
 - Render Markdown strings dynamically into HTML nodes using custom helpers.
 - In the Quiz controller, render question sets, track selected index positions, calculate quiz scores, display explanations, and compile payload strings to request ReportLab PDF generation.
- Viewport Scrolling:
 - Call `.scrollIntoView({ behavior: 'smooth' })` to automatically scroll the user's viewport to the newly generated results.

3. UI State Cleanup and Reset

- State Restorations:
 - In the finally block of the Fetch request handlers, re-enable the submit buttons and restore original texts, icons, and page loaders regardless of request success or failure.

- Feedback Notifications (toast.js):
 - Display custom visual popup notifications (success/warning alerts) to indicate success states (e.g. "Text Copied to Clipboard!" or "Document Upload Completed!")



Milestone 5: Deployment

In Milestone 5, the focus is on deploying the EduGenie application locally to verify correct operation of all modular features, and optionally exposing the local server publicly via Ngrok to share the application over a secure public URL without cloud hosting.

Activity 5.1: Preparing the Application for Local Deployment

1. Set Up a Virtual Environment:

- Open a new Command Prompt (or PowerShell terminal) and navigate to the project root directory.
- Create and activate a Python virtual environment to isolate the project dependencies:

```
python -m venv venv  
venv\Scripts\activate.bat
```

- Install all required dependencies from the requirements.txt file:

```
pip install -r requirements.txt
```

2. Configure Environment Variables:

- Create a .env file in the project root directory.
- Populate it with your secure API tokens. The FastAPI application loads these configurations automatically at startup:

GEMINI_API_KEY=your_actual_google_studio_gemini_api_key_here

SECRET_KEY=your_generated_cookie_signing_secret_here

HOST=0.0.0.0

PORT=8000

DEBUG=true

Activity 5.2: Local Testing and Verification

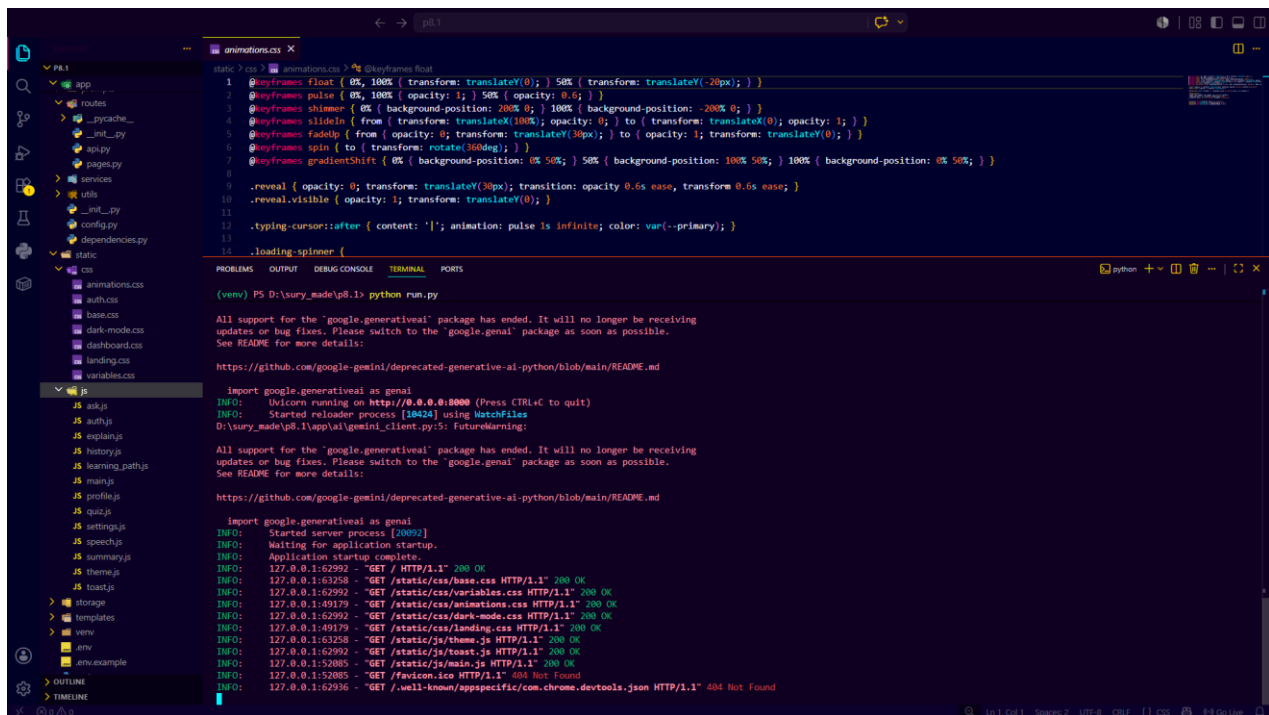
1. Start the FastAPI Application Server:

- Run the application launcher script:

```
python run.py
```

(This launches Uvicorn to run the app, listening on `http://0.0.0.0:8000` with hot-reloading enabled in debug mode).

In the place of 0.0.0.0 please use localhost



The screenshot shows a VS Code editor with a file explorer on the left containing files like `app.py`, `static/css/`, and `static/js/`. The main editor displays the `animations.css` file with several keyframe animations. Below the editor, the **TERMINAL** tab is active, showing the command `python run.py` and the output of the Uvicorn server. The output indicates that the server is running on `http://0.0.0.0:8000` and lists the files being served, such as `static/css/base.css`, `static/css/variables.css`, and `static/js/main.js`.

- Open a modern web browser and navigate to `http://127.0.0.1:8000` to access the EduGenie web interface.

2. Verify AI Educational Core Features:

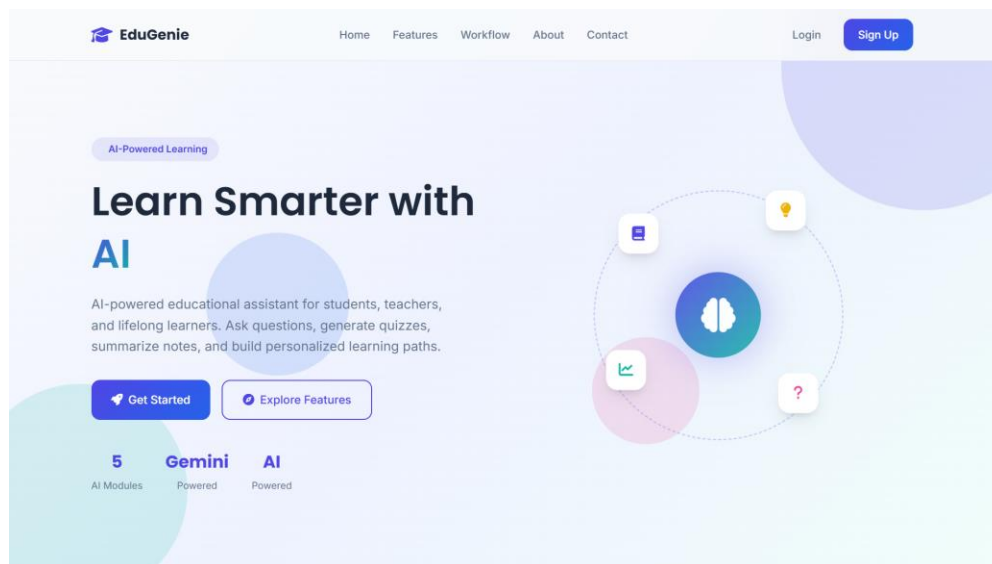
- Register a new user and log in to access the student dashboard.
- Verify cumulative analytics card updates (streak tracking, XP, badges).

- Test the Q&A / RAG Document Chat by uploading a PDF and asking context-specific questions.
 - Verify the Concept Explainer scales explanations across Beginner, Intermediate, and Advanced levels.
 - Play a generated quiz and verify you can download the quiz results as a PDF.
 - Verify all study histories, settings, and bookmarks are correctly logged in the local JSON database folder under storage/data/.
-

Exploring the Website's Web Pages:

Home / Generator Page:

The public landing page serves as the entry portal for visitors. It introduces the platform's core AI modules including the RAG Chat, Concept Explainer, Quiz Generator, Summarizer, and Learning Paths. The page features a modern layout with clean call-to-action buttons directing users to sign up or log in.



User Authentication Pages (Login & Signup):

These pages present clean, centered glassmorphic login and signup cards displayed on an animated blob background. The authentication forms manage user registration and login validations, check email formatting rules, enforce password matching constraints, and automatically redirect authorized users to the dashboard home page.

LOGIN PAGE:


EduGenie

Learn Smarter with AI

Your personal AI tutor for questions, quizzes, summaries, and personalized learning paths.

- AI-powered explanations
- Interactive quiz generator
- Personalized roadmaps

5 AI Tools
 24/7 Available
 Free to Start

Welcome back

Sign in to continue your learning journey

Email address

Password

☐ Remember me
 [Forgot password?](#)

[Sign In →](#)

or

Don't have an account? [Create one free](#)

SIGN UP PAGE:


EduGenie

Start Your Learning Journey

Join thousands of students using AI to study smarter, not harder.

- ★ Ask AI any question
- ★ Generate practice quizzes
- ★ Track your progress

"EduGenie helped me understand complex topics in minutes!"

— Student, Computer Science

Create account

Free forever. No credit card required.

Username

Email address

Password

Password strength

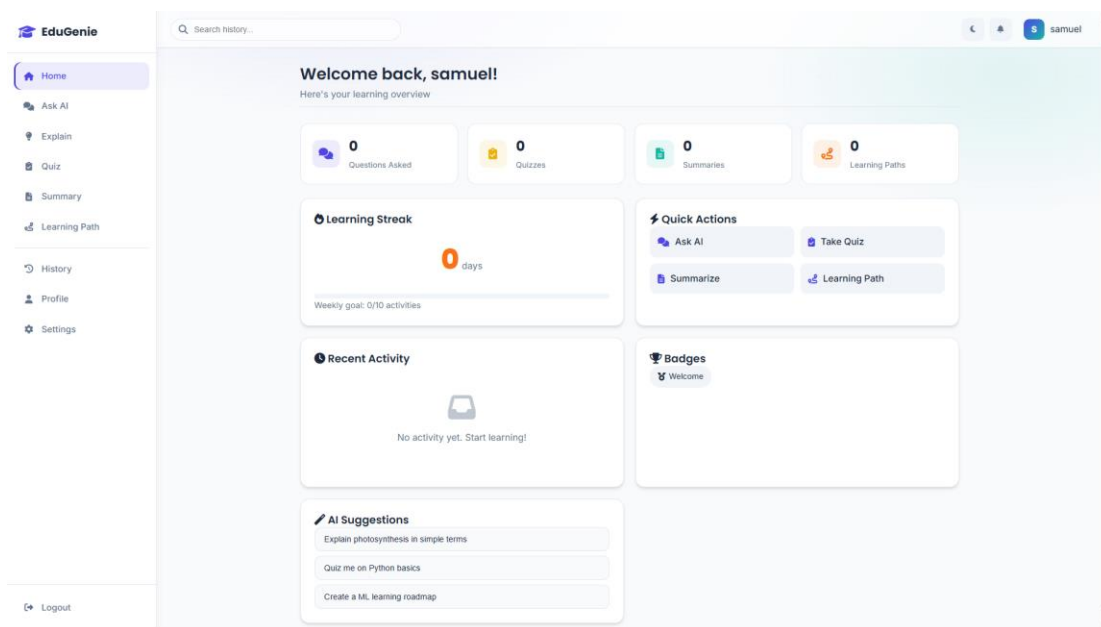
Confirm password

[Create Account →](#)

Already have an account? [Sign in](#)

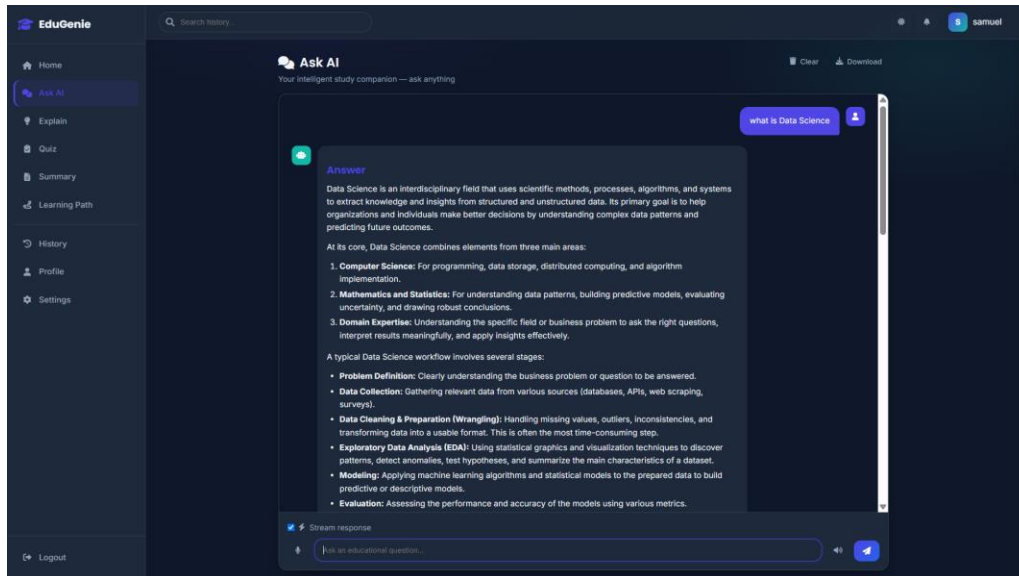
Main Dashboard Home (Analytics & Streaks):

The central landing area for authenticated students provides an overview of their learning progress. It displays cumulative study metrics across four statistic cards tracking total Questions Asked, Quizzes Taken, Summaries Created, and Learning Paths generated. The view also integrates a gamified panel showing the student's current active daily study streak, accumulated Experience Points (XP), and unlocked study achievement badges.



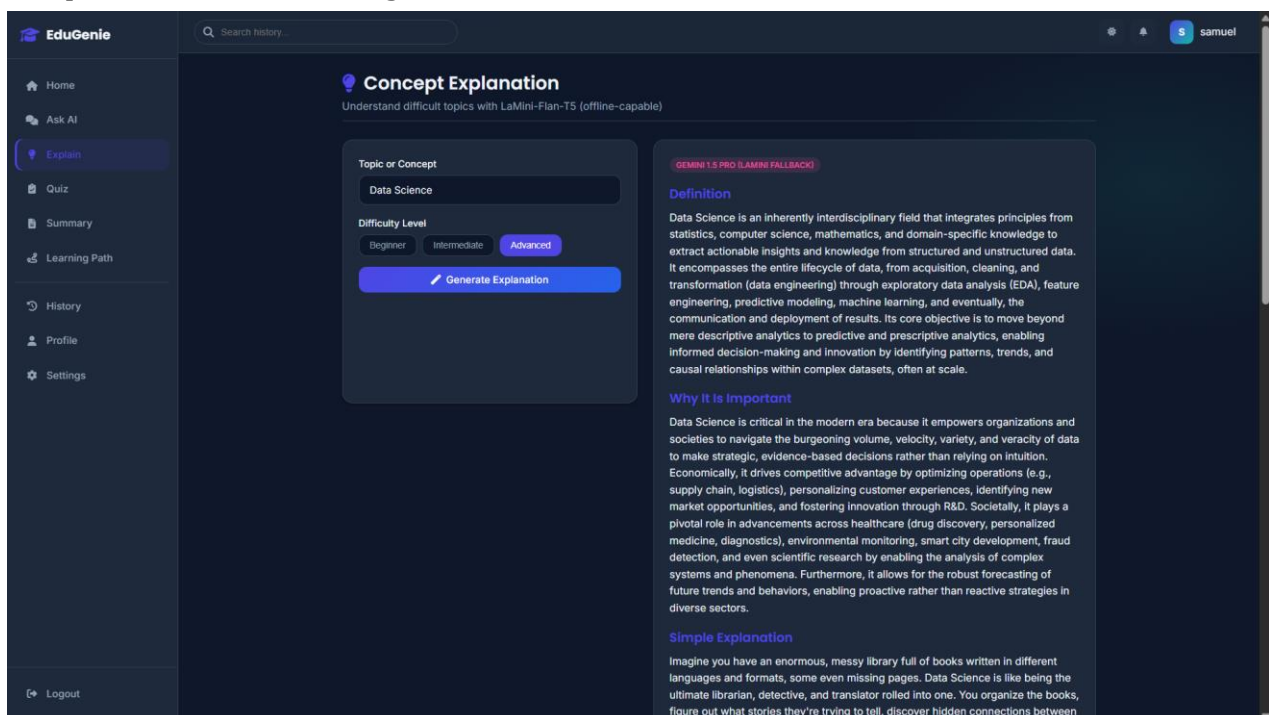
Conversational Q&A / RAG Document Chat Page:

This dual-purpose learning chat page contains a main messaging interface where user prompts are sent and AI replies are streamed back chunk-by-chunk. The interface supports voice-to-text transcription and text-to-speech voice playback. A sliding document drawer on the right side lets users upload PDF or DOCX textbooks to enable RAG mode, allowing them to ask questions answered solely using the document's text context.



Scaled Concept Explainer Page:

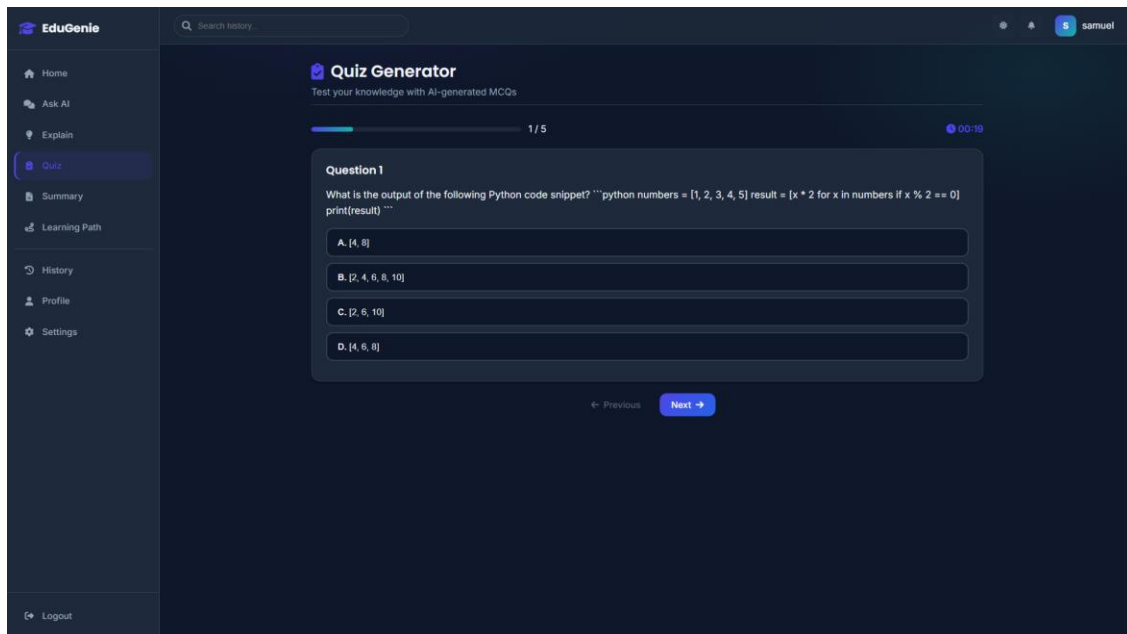
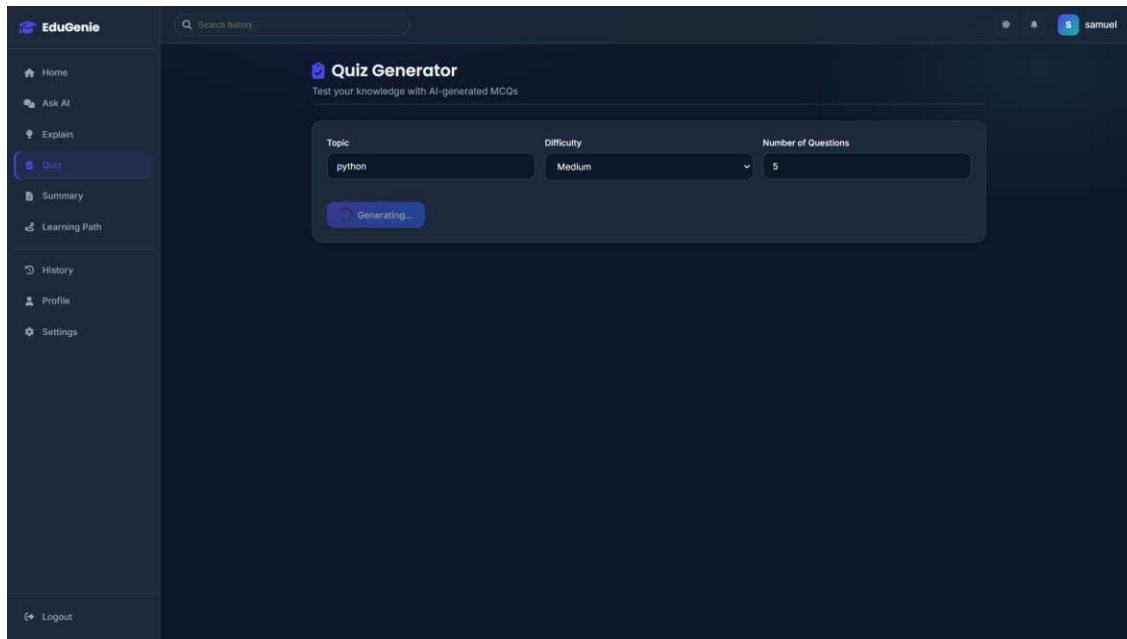
The concept explainer panel allows students to enter a topic and select from three difficulty levels: Beginner, Intermediate, and Advanced. The explainer processes the request and renders scaled, formatted explanations containing real-world analogies, code blocks, and comparative tables matching the selected student level.



Interactive Quiz Generator & Player Page:

The quiz interface lets students configure custom quizzes by topic, difficulty, and number of questions. Once generated, the app transitions into an interactive Quiz Player screen displaying multiple-choice questions, option selectors, and a counting quiz timer. Submitting the answers renders a scoreboard showing correct answers, detailed explanations, and an option to export the quiz to a PDF document.

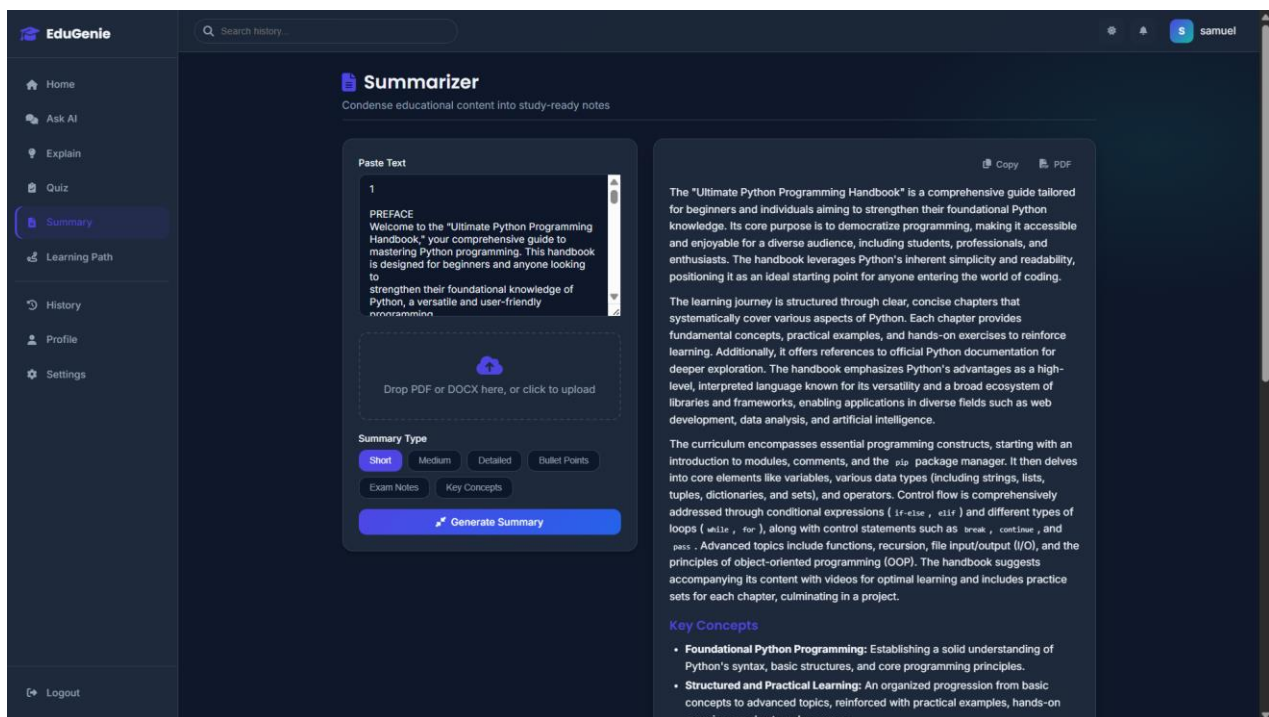
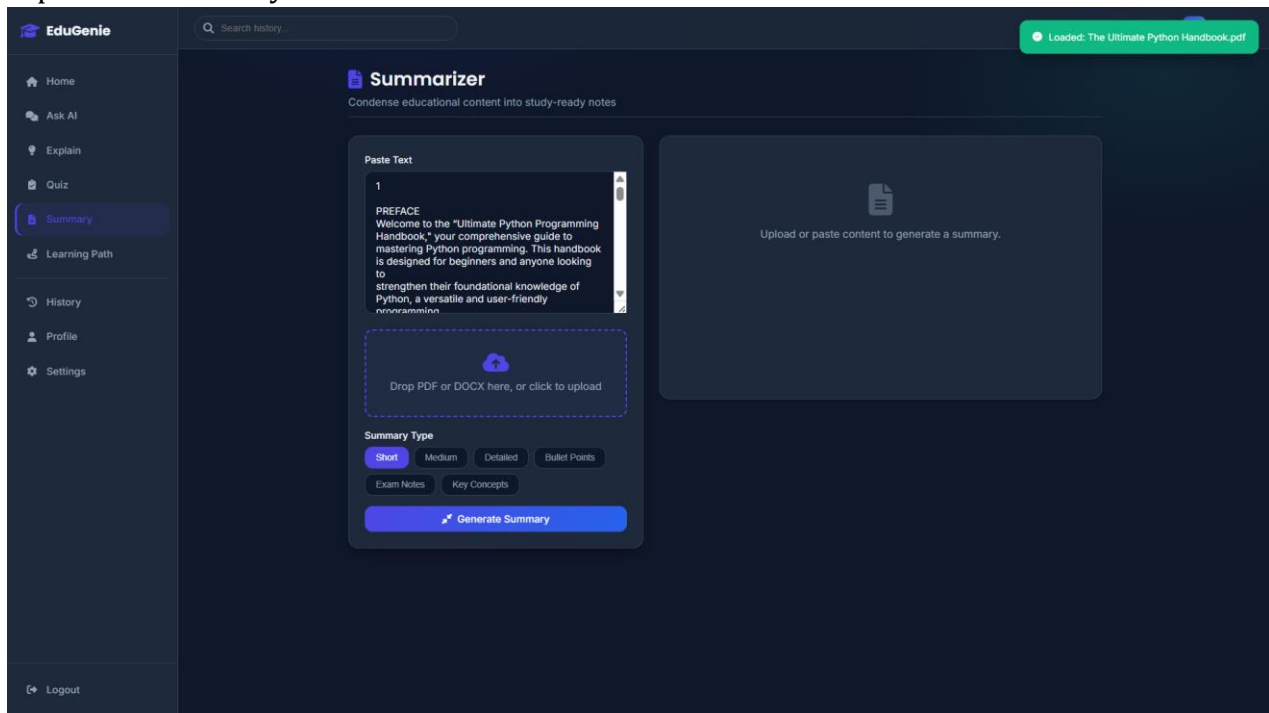
Process:



Note Summarizer Page:

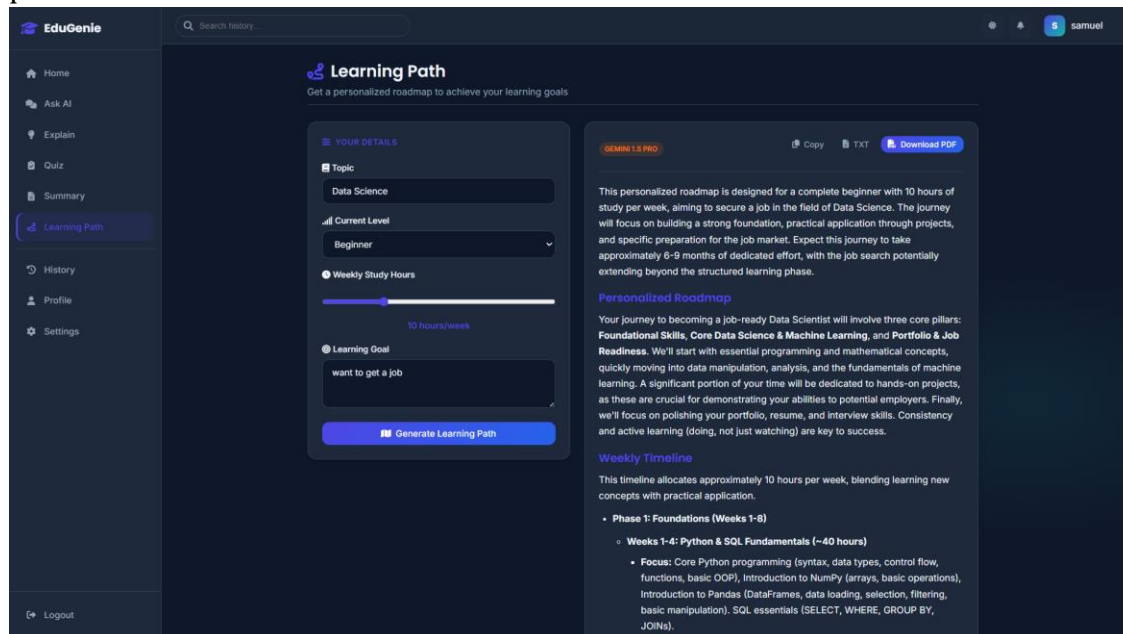
This document processor page accepts raw text inputs or uploaded PDF and DOCX files.

Users can choose to generate a concise bulleted summary or detailed study notes. The panel displays the formatted Markdown notes with options to copy summaries to the clipboard or export them directly as a PDF document.



Personalized Learning Paths Page:

The roadmap generator takes a topic, current level, weekly study hours, and target learning goals as inputs. It outputs a structured, week-by-week timeline roadmap detailing study modules, subtopics, resources, and tasks, alongside a download action to export the study plan as a PDF document.



EduGenie Search history

Learning Path
Get a personalized roadmap to achieve your learning goals

YOUR DETAILS

- Topic: Data Science
- Current Level: Beginner
- Weekly Study Hours: 10 hours/week
- Learning Goal: want to get a job

Generate Learning Path

GEMINI 1.5 PRO Copy TXT Download PDF

This personalized roadmap is designed for a complete beginner with 10 hours of study per week, aiming to secure a job in the field of Data Science. The journey will focus on building a strong foundation, practical application through projects, and specific preparation for the job market. Expect this journey to take approximately 6-9 months of dedicated effort, with the job search potentially extending beyond the structured learning phase.

Personalized Roadmap

Your journey to becoming a job-ready Data Scientist will involve three core pillars: **Foundational Skills**, **Core Data Science & Machine Learning**, and **Portfolio & Job Readiness**. We'll start with essential programming and mathematical concepts, quickly moving into data manipulation, analysis, and the fundamentals of machine learning. A significant portion of your time will be dedicated to hands-on projects, as these are crucial for demonstrating your abilities to potential employers. Finally, we'll focus on polishing your portfolio, resume, and interview skills. Consistency and active learning (doing, not just watching) are key to success.

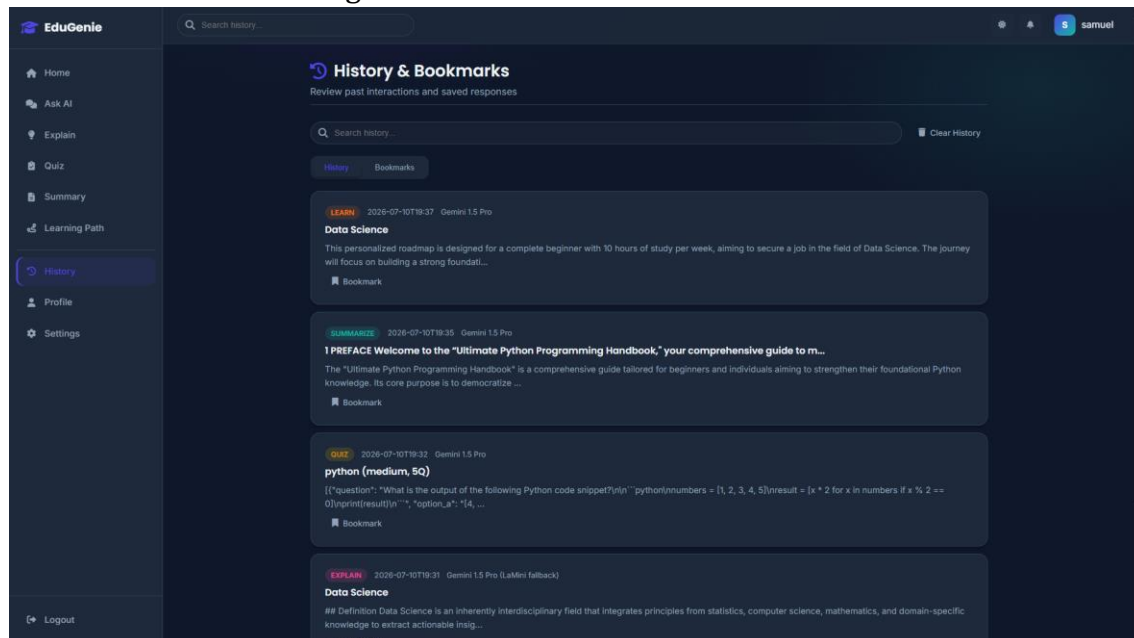
Weekly Timeline

This timeline allocates approximately 10 hours per week, blending learning new concepts with practical application.

- Phase 1: Foundations (Weeks 1-8)**
 - Weeks 1-4: Python & SQL Fundamentals (~40 hours)**
 - Focus:** Core Python programming (syntax, data types, control flow, functions, basic OOP), Introduction to NumPy (arrays, basic operations), Introduction to Pandas (DataFrames, data loading, selection, filtering, basic manipulation). SQL essentials (SELECT, WHERE, GROUP BY, JOINS).

Query History & Bookmarks Page:

The history log page displays a list of all previous AI queries, answers, and the models used. The view includes a text search filter to quickly scan past studies, query deletion actions, and a Bookmarks tab showing saved educational references.



EduGenie Search history

History & Bookmarks
Review past interactions and saved responses

Search history... Clear History

History Bookmarks

LEARN 2026-07-10T19:37 Gemini 1.5 Pro
Data Science
This personalized roadmap is designed for a complete beginner with 10 hours of study per week, aiming to secure a job in the field of Data Science. The journey will focus on building a strong foundati...
Bookmark

SUMMARIZE 2026-07-10T19:35 Gemini 1.5 Pro
1 PREFACE Welcome to the "Ultimate Python Programming Handbook," your comprehensive guide to m...
The "Ultimate Python Programming Handbook" is a comprehensive guide tailored for beginners and individuals aiming to strengthen their foundational Python knowledge. Its core purpose is to democratize ...
Bookmark

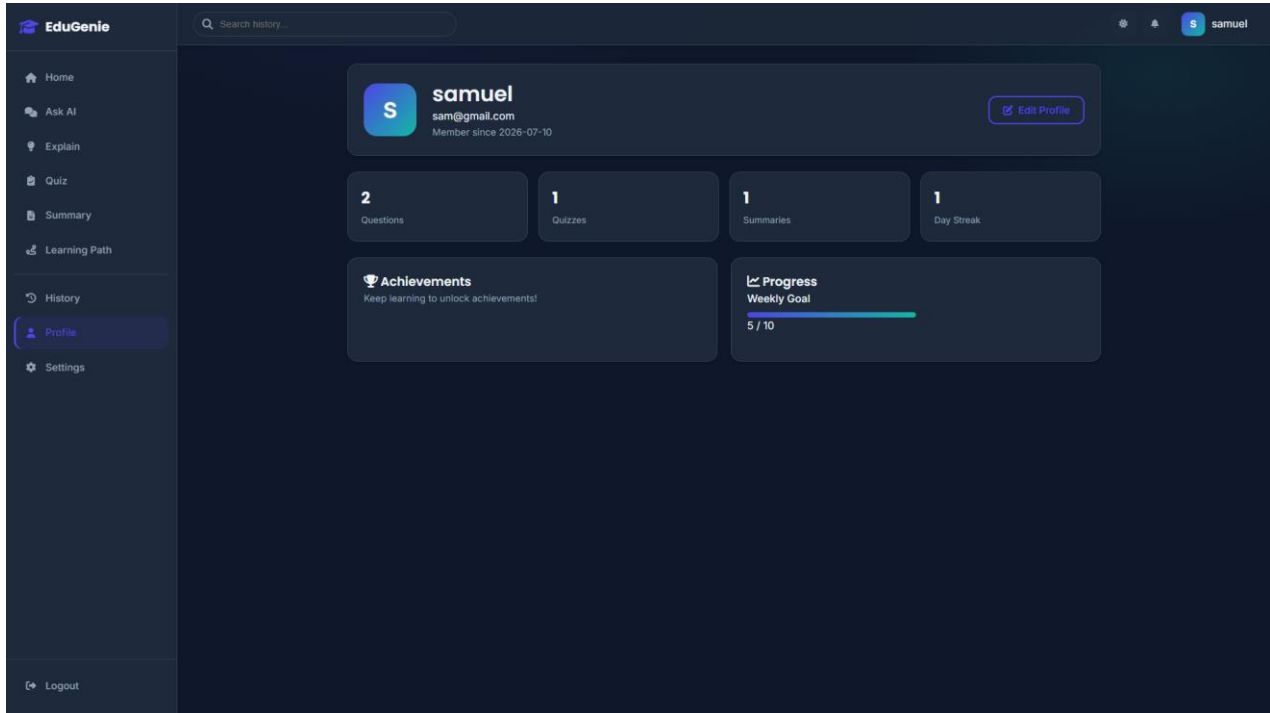
QUIZ 2026-07-10T19:32 Gemini 1.5 Pro
python (medium, 5Q)
[{"question": "What is the output of the following Python code snippet?\\n\\npython\\nnumbers = [1, 2, 3, 4, 5]\\nresult = [x * 2 for x in numbers if x % 2 == 0]\\nprint(result)\\n\\n", "option_a": "14, ...
Bookmark

EXPLAIN 2026-07-10T19:31 Gemini 1.5 Pro (LaMini fallback)
Data Science
Definition Data Science is an inherently interdisciplinary field that integrates principles from statistics, computer science, mathematics, and domain-specific knowledge to extract actionable insig...
Bookmark

User Profile & Settings Page:

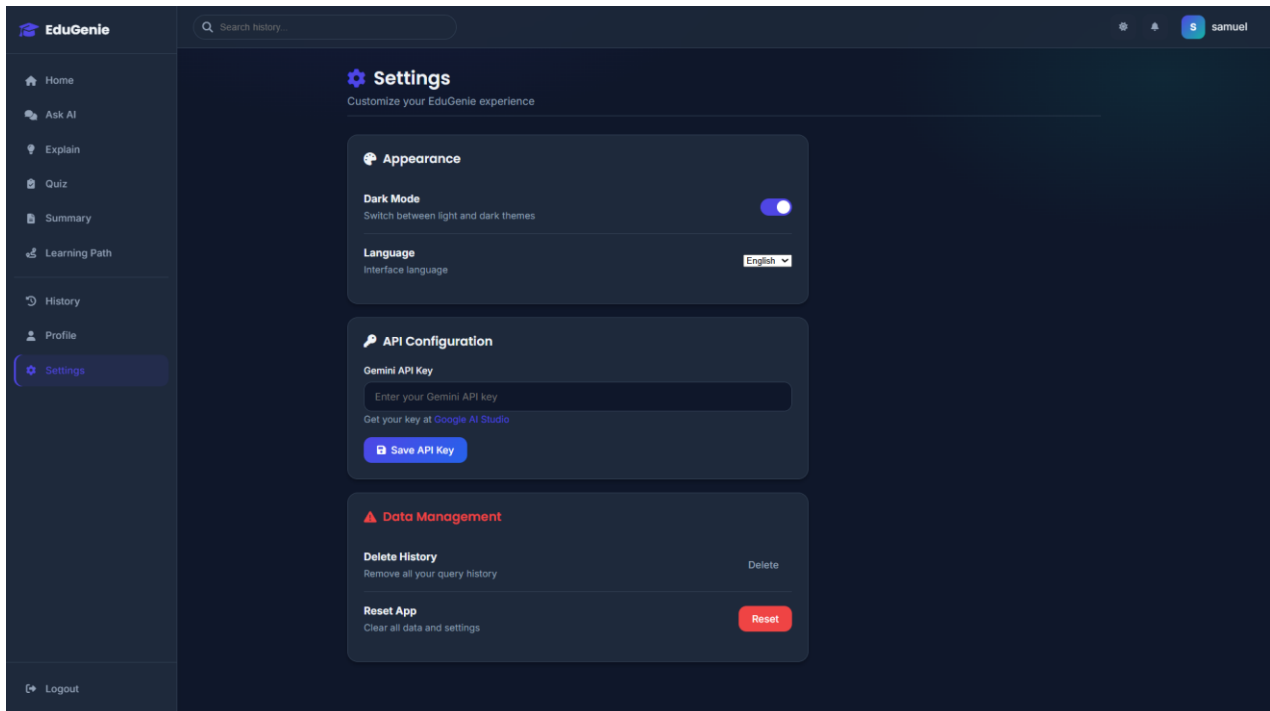
The settings page allows users to update their profile username and email address. It features settings forms to toggle between light and dark UI themes, set language preferences, and securely save a custom Google Gemini API Key.

PROFILE:



The screenshot shows the 'Profile' page of the EduGenie application. The left sidebar contains navigation links: Home, Ask AI, Explain, Quiz, Summary, Learning Path, History, Profile (selected), and Settings. The main content area displays the user's profile for 'samuel' (sam@gmail.com, Member since 2026-07-10) with an 'Edit Profile' button. Below the profile are four statistics: 2 Questions, 1 Quizzes, 1 Summaries, and 1 Day Streak. There are also sections for 'Achievements' (Keep learning to unlock achievements!) and 'Progress' (Weekly Goal 5 / 10).

SETTINGS:



The screenshot shows the 'Settings' page of the EduGenie application. The left sidebar is the same as the profile page, with 'Settings' now selected. The main content area is titled 'Settings' with the subtitle 'Customize your EduGenie experience'. It contains three sections: 'Appearance' with a 'Dark Mode' toggle (switched on) and a 'Language' dropdown (set to English); 'API Configuration' with a 'Gemini API Key' input field, a link to 'Get your key at Google AI Studio', and a 'Save API Key' button; and 'Data Management' with a 'Delete History' button (Remove all your query history) and a 'Reset App' button (Clear all data and settings).

Milestone 6:

Conclusion & Future Scope:

6.1 Conclusion:

EduGenie is a production-ready, full-stack generative AI learning assistant built with FastAPI and styled with a responsive glassmorphism user interface that streamlines study management for students, teachers, and self-learners. By integrating cloud-based reasoning via the official Google Gemini API and a local, offline LaMini-Flan-T5 model, the project demonstrates a hybrid AI approach that balances fast cloud inference with local, privacy-respecting offline features. The successful implementation of RAG-driven document chat, scaled concept explanations, interactive quiz players, text summarizers, and timeline roadmaps highlights how targeted AI models, when combined with a clean and structured framework, can boost study productivity and user engagement. The project also demonstrates how file parsers, local flat JSON databases, JWT cookie-based session security, and vector similarity calculations can be implemented in a lightweight footprint without the overhead of heavy external servers. The gamification mechanics—including active daily streak tracking, XP accrual, and milestone badge awards—encourage consistent interaction and help gamify the student learning experience. Combined with automatic PDF document exports generated on-the-fly using ReportLab, the application transitions from a simple AI tool into a cohesive educational dashboard.

6.2 Future Scope:

Looking ahead, EduGenie offers significant opportunities for future expansion and scalability. Future iterations could integrate dedicated local vector databases such as ChromaDB or FAISS to persist and query large multi-document libraries more efficiently. Implementing full OAuth2 multi-user registration, database encryption, and secure cloud database persistence (such as PostgreSQL) would enable scaling the platform to support classroom deployments. Additionally, leveraging Gemini's multimodal capabilities to analyze images and mathematical formulas in textbook uploads, alongside implementing automated quiz recommendations based on historical user performance, would position EduGenie as a highly adaptive and comprehensive digital tutor.